

MODULE 3

Data Modeling with MongoDB

Design documents around how the application reads, writes and grows — then build training_store.

**Documents and
types**

Access patterns

Embed or reference

Schema patterns

Validate and build

LEARNING OBJECTIVES

What you will be able to do by the end of this module.

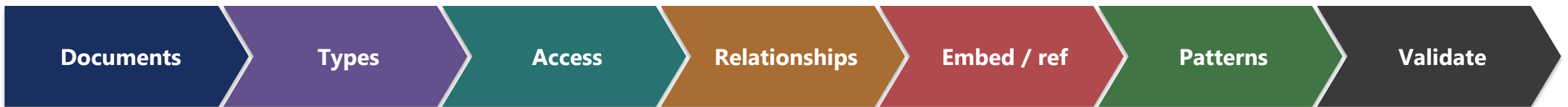


BY THE END OF THIS MODULE, YOU CAN

1. Read the structure of a MongoDB document
2. Select the right BSON type for each value
3. Identify the application's access patterns
4. Model relationships by embedding or referencing
5. Keep documents and arrays bounded
6. Apply schema patterns and validation
7. Build and query training_store



MODULE ROADMAP



SIX QUESTIONS THIS MODULE ANSWERS

WHAT

is inside a document?

WHICH

type fits each value?

HOW

does the application use the data?

WHEN

do we embed — or reference?

WHICH

patterns and pitfalls matter?

HOW

do we enforce and build the model?

KEY TAKEAWAY Shape each document around how the application uses it — then justify every choice.

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 Access pattern

One read or write the application runs often — it decides the document shape.

3 Referencing

Storing another document's `_id` instead of its data, like `customerId` in an order.

5 Bounded array

An array with a known, small maximum size — an order's line items, not every review.

7 Schema pattern

A reusable modeling solution, such as Attribute, Subset, Computed or Bucket.

2 Embedding

Storing related data inside the parent document, like addresses inside a customer.

4 Cardinality

How many items sit on each side of a relationship: one, a few, many or millions.

6 Snapshot

A copy taken at one moment and never synced, like the price paid on an order.

8 Schema validation

`$jsonSchema` rules on a collection that check required fields and types on writes.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

BSON

Binary JSON

ISBN

International Standard Book Number

SKU

Stock Keeping Unit

CAD

Canadian Dollar

JSON

JavaScript Object Notation

SQL

Structured Query Language

CPU

Central Processing Unit

MiB

Mebibyte (1,048,576 bytes)

USB

Universal Serial Bus

ID

Identifier

RAM

Random Access Memory

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

FROM MODULE 2 TO MODULE 3

MongoDB is running — now we decide what the documents should look like.

MODULE 2 GAVE US

- ✓ A running MongoDB — local or Atlas
- ✓ mongosh connected and verified
- ✓ use, createCollection and insertOne
- ✓ A first starter products collection
- ✓ Fixes for common connection errors

NOW Design the documents before we insert many more of them.



OUR PATH THROUGH MODULE 3

- 1 Documents, BSON types and a shared core
- 2 Access patterns, relationships and growth
- 3 Embed, reference and snapshots
- 4 Anti-patterns and schema patterns
- 5 Validation, evolution and the labs

Part 1

Part 2

Part 3

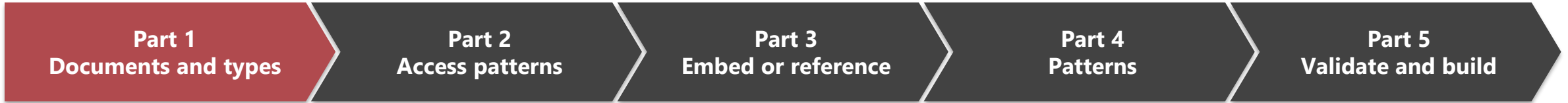
Part 4

Part 5

KEY TAKEAWAY Module 2 gave us a working database; Module 3 decides the shape of everything we store in it.

PART 1: DOCUMENTS AND BSON TYPES

Start with what a document can hold.



SO FAR

- ✓ MongoDB is installed and mongosh connects
- ✓ training_store has its first collection

NEXT QUESTION

What can one document hold — and which type should each value have?

IN THIS PART

- 1 Anatomy of a training_store document
- 2 Nested documents and arrays
- 3 Common BSON types
- 4 Choosing types that match the meaning
- 5 One collection, a shared core

ANATOMY OF A TRAINING_STORE DOCUMENT

Product L100, field by field — every value keeps its BSON type.

`db.products.findOne({ sku: "L100" })`

```
{
  _id: ObjectId("..."),           // ObjectId
  sku: "L100",                     // String
  name: "Business Laptop",
  category: "LAPTOP",
  price: Decimal128("1299.99"),    // Decimal128
  attributes: {                    // embedded document
    processor: "Intel Core i7",
    memoryGB: 16, storageGB: 512    // Int32
  },
  tags: ["business", "portable"],  // Array
  active: true,                     // Boolean
  createdAt: ISODate("2026-08-20T09:00:00Z") // Date
}
```

_id rules

Unique in its collection, indexed automatically, and can't be changed after insert.

Any unique value

ObjectId by default — a string or a number works too, if it stays unique.

Business key

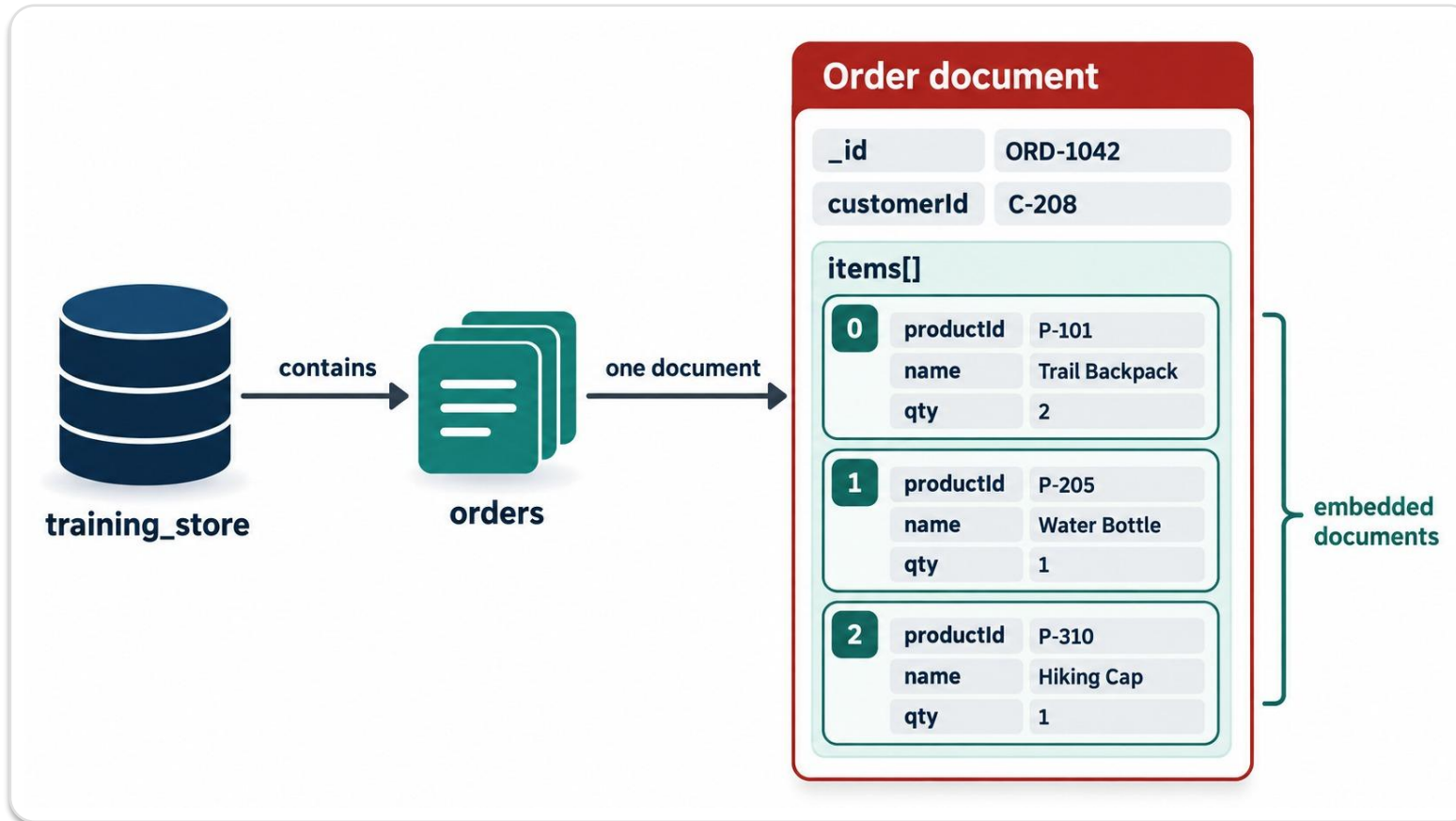
sku is a separate field with its own unique index — it is what people search by.

EXERCISE 3.1 You'll label a customer document the same way.

KEY TAKEAWAY A document is a set of typed field-value pairs — and `_id` is the one field every document must have.

NESTED DOCUMENTS AND ARRAYS

A value can be a whole document — or a list of them.



Embedded document

A document as a value: a product's attributes, a customer's name and contact.

Array

Several values in one field: tags, or whole documents like order items and addresses.

Reach inside with dot notation

```
db.products.find(
  { "attributes.memoryGB": 16 })
db.orders.find(
  { "items.sku": "L100" })
```

QUOTES Dotted paths must be in quotes.

KEY TAKEAWAY Nested documents and arrays let one document hold a whole business object — dot notation reaches inside.

COMMON BSON TYPES

Each type with a real training_store value.

Type	training_store example	Typical use
String	sku: "L100"	Names, codes, statuses
Int32	memoryGB: 16	Quantities, counters
Decimal128	price: Decimal128("1299.99")	Money
Double	4.6 (an average)	Measurements
Boolean	active: true	Flags
Date	createdAt: ISODate("2026-08-20...")	Timestamps
ObjectId	customerId: ObjectId("...")	Identifiers
Array	tags: ["business", "portable"]	Lists
Embedded	attributes: { processor: ... }	Grouped fields
Null	contact.phone: null (C204)	Known but empty

Int32 or Double?

mongosh stores a whole number like 16 as Int32 and 4.6 as a Double.

Null or missing?

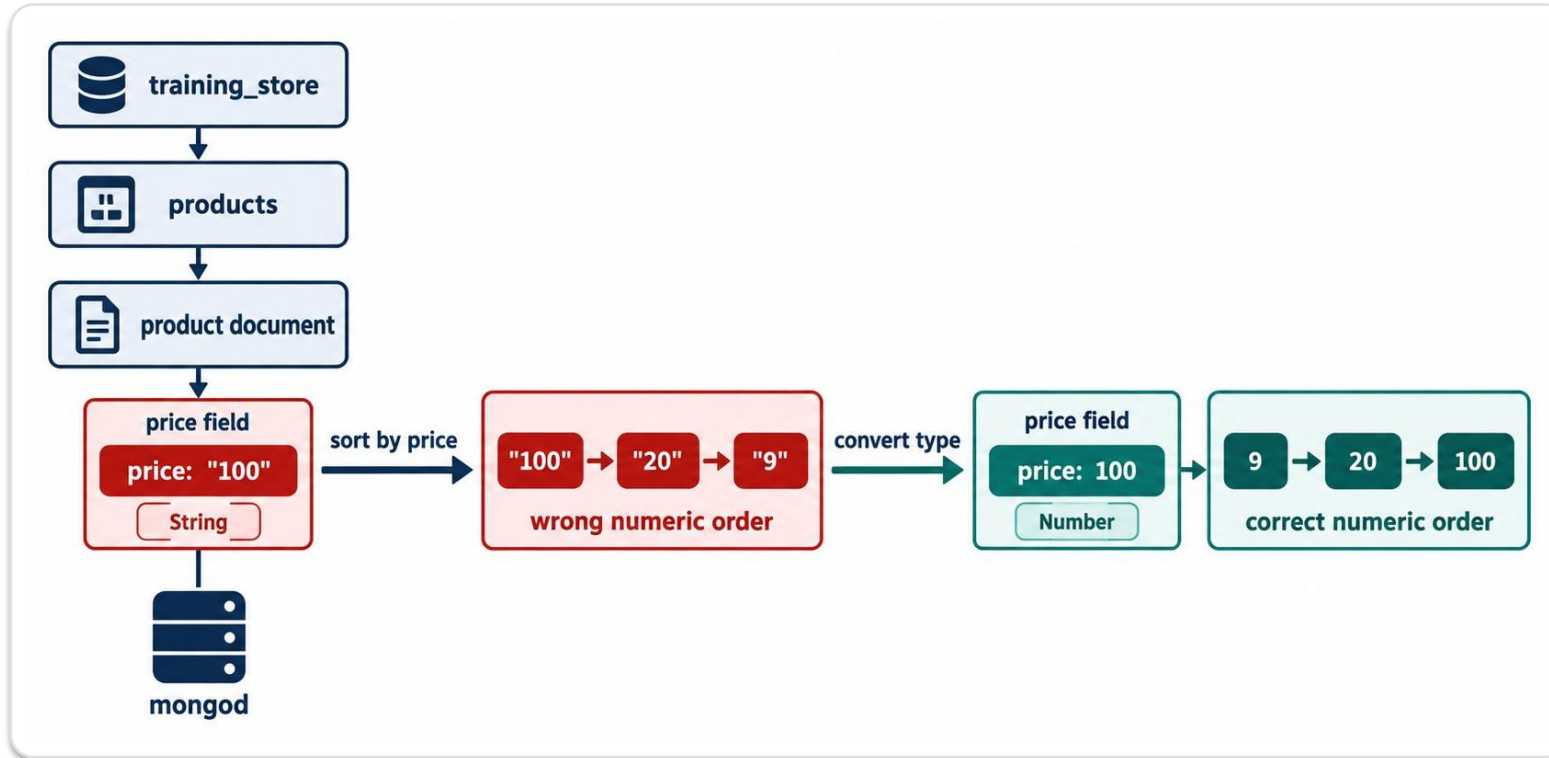
C204's phone is null; C515 has no phone field at all.

MONEY Decimal128 — never a string or a Double.

KEY TAKEAWAY BSON adds the types JSON lacks — dates, decimals and ObjectIds — so choose the one that matches the meaning.

CHOOSE TYPES THAT MATCH THE MEANING

Strings make comparisons, sorting and calculations wrong.



Poor versus better

```
// poor
{ price: "49.99", active: "yes",
  createdAt: "September 20, 2026" }
// better
{ price: Decimal128("49.99"),
  active: true,
  createdAt: ISODate("2026-09-20") }
```

Why it matters

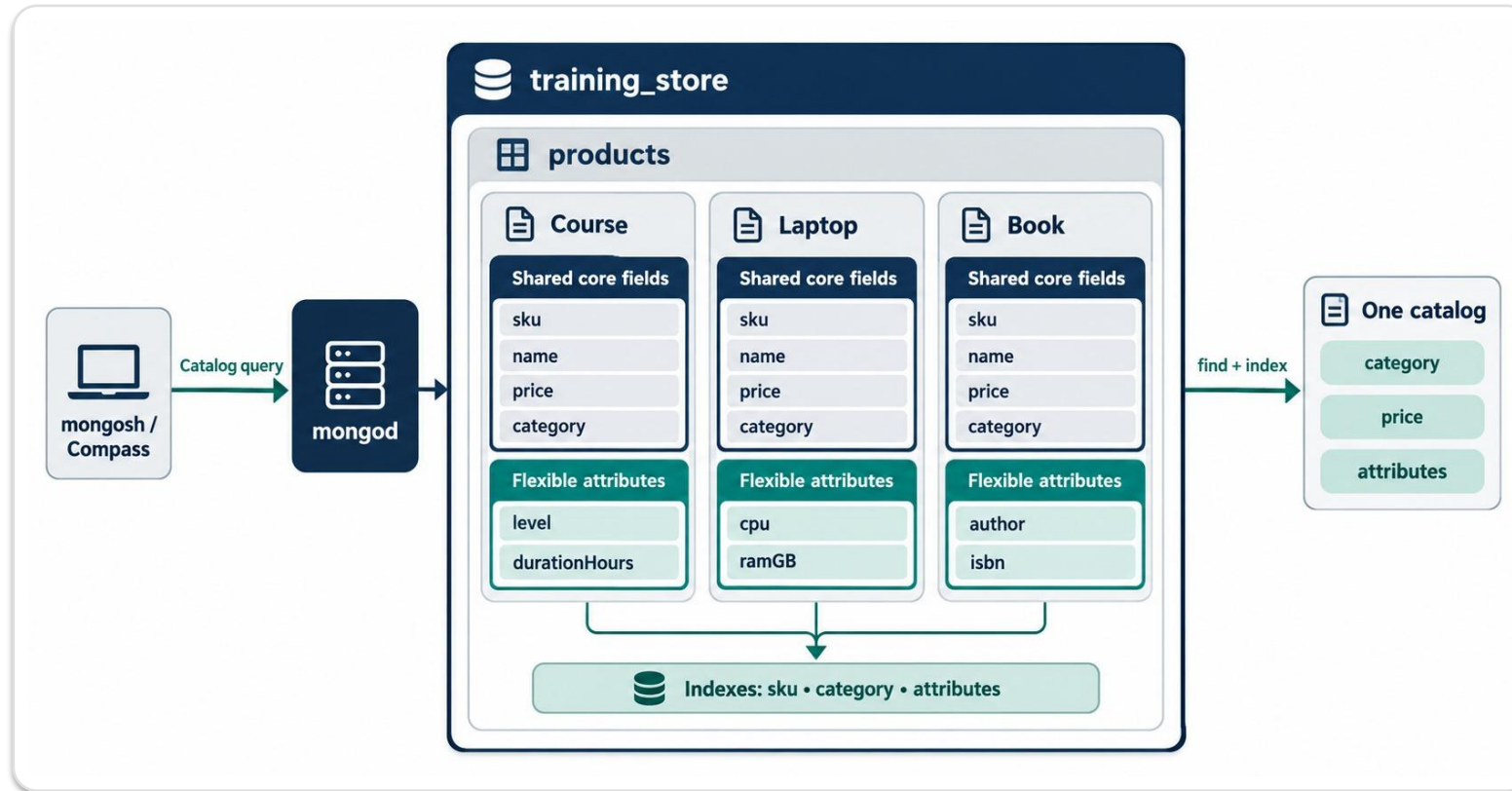
"100" sorts before "20"; "yes" is not a Boolean; text dates don't sort by time.

IN TRAINING_STORE XBAD's price is the string "49.99" — on purpose.

KEY TAKEAWAY Store money as Decimal128, flags as Booleans and times as Dates — the type decides how values behave.

ONE COLLECTION, A SHARED CORE

Different products, the same core fields, flexible attributes.



Shared core

sku, name, category, price, tags, active, createdAt — same names, same types.

Flexible attributes

LAPTOP: processor, memoryGB · SHOE: sizes, color · BOOK: author, isbn.

One name per field

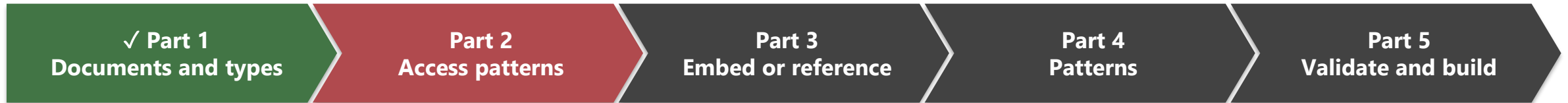
name everywhere — never productName on one product and title on another.

EXERCISE 3.4 You'll design this catalog yourself.

KEY TAKEAWAY Keep the core identical across products and put category-specific fields under attributes.

PART 2: DESIGNING AROUND ACCESS PATTERNS

Flexible doesn't mean unplanned.



SO FAR

- ✓ A document holds typed fields, nested documents and arrays
- ✓ Choose each BSON type by its meaning
- ✓ One collection can hold varied products

NEXT QUESTION

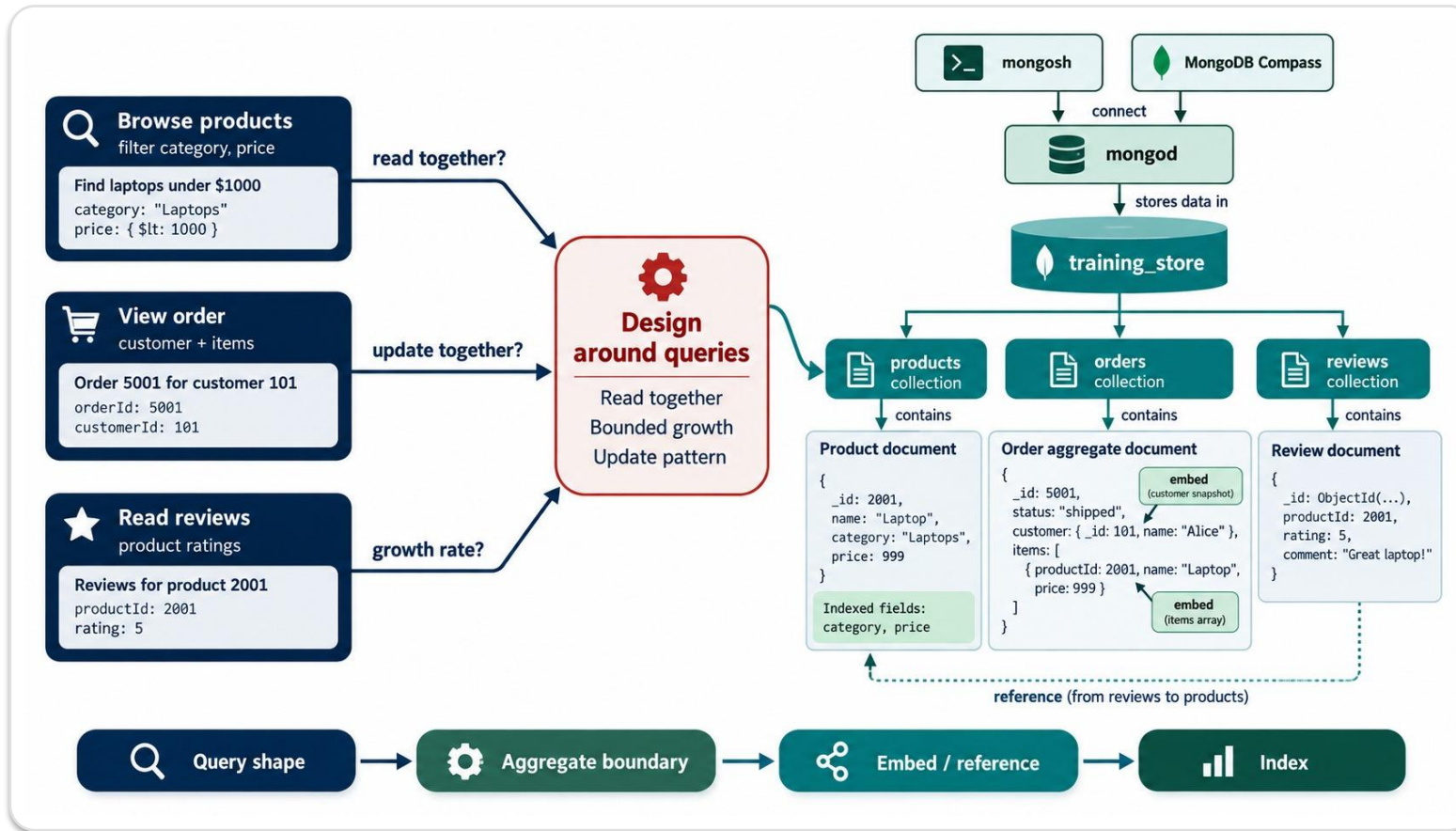
How does the application actually use the data — and how should that shape the documents?

IN THIS PART

- 1 Model around access patterns
- 2 training_store's access patterns
- 3 Relationship types
- 4 Estimating growth: bounded or not

MODEL DATA AROUND ACCESS PATTERNS

Start from the operations — not from the tables.



Operations first

Browse products, view an order, read reviews — before any collection exists.

Three questions

Read together? Updated together? How fast does it grow?

Then shape and index

Aggregate boundary → embed or reference → indexes.

PRINCIPLE Data accessed together is stored together.

KEY TAKEAWAY The application's most important operations decide the document shape — design around the queries.

TRAINING_STORE ACCESS PATTERNS

The operations that shape our four collections.

Access pattern	Frequency	Returned together	Modeling implication
Find product by SKU	High	The whole product	One product document
Browse category and price	High	Name, price	Shared core fields
Show customer profile	High	Name, contact, addresses	Embed addresses
Show one order	High	Items, totals, address	Embed items, snapshot
Customer order history	Medium	Order summaries	customerId on each order
Reviews for a product	Medium	Recent reviews	Separate collection
Change order status	Medium	One order	One-document update

Entities

Product, Customer, Order, Review.

Relationships

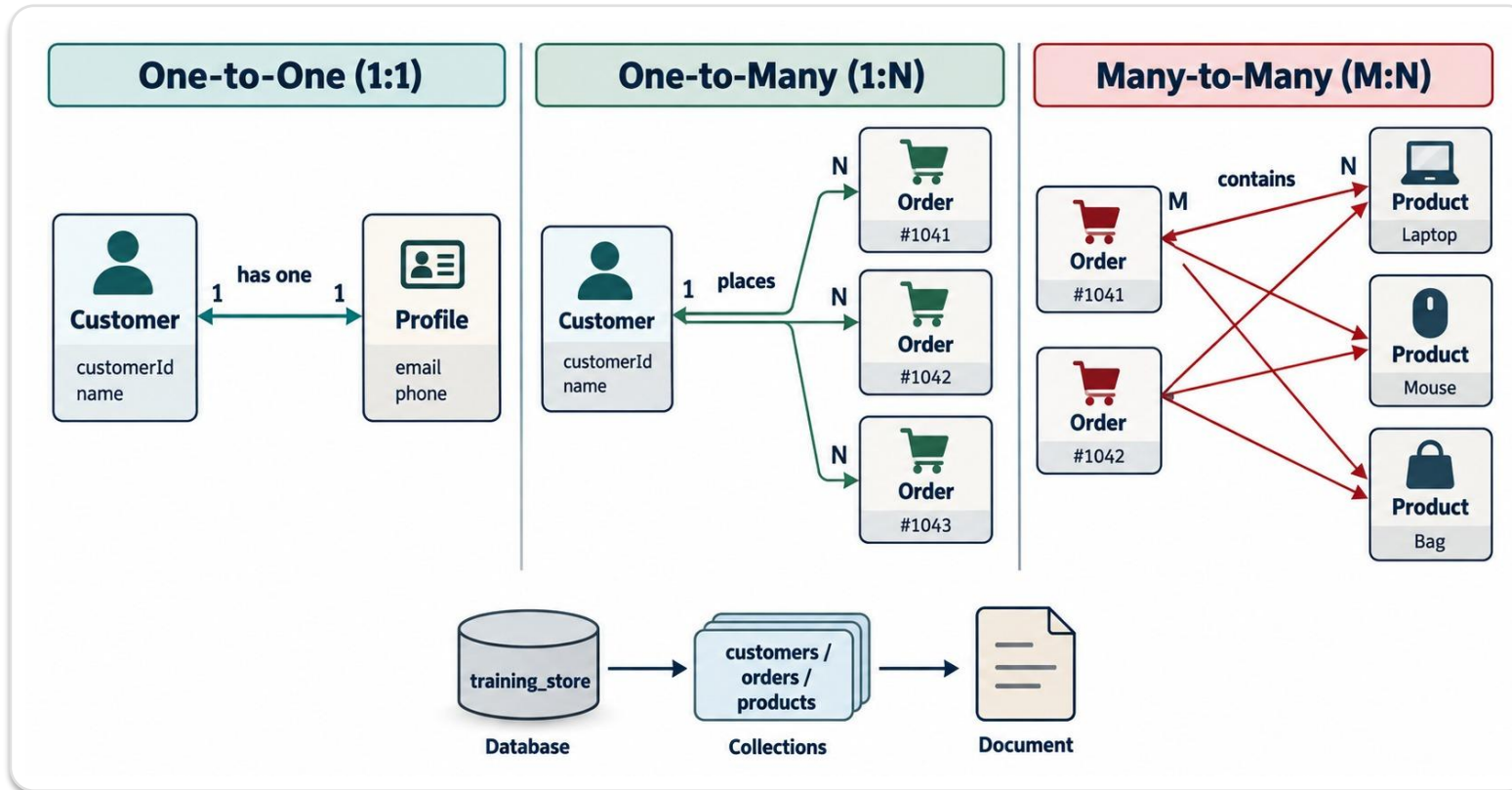
Customer → orders and reviews;
product → reviews; order → items.

EXERCISE 3.2 You'll build this table from the requirements.

KEY TAKEAWAY List each operation with how often it runs and what it returns together — the implications follow.

RELATIONSHIP TYPES

One-to-one, one-to-many and many-to-many.



One-to-one

Customer → preferences: embedded as one sub-document.

One-to-many

Customer → orders: each order stores customerId.

Many-to-many

Orders ↔ products: linked through items[].productId.

ALSO ASK How many on the many side — a few or millions?

KEY TAKEAWAY Name each relationship's type, then count the many side before choosing a model.

ESTIMATE GROWTH: BOUNDED OR UNBOUNDED?

Small arrays can be embedded — growing ones can't.

BOUNDED ARRAY

 training_store • products

```
{
  _id: "P204",
  name: "MongoDB Basics",
  featuredReviews: [
    { _id: "R901", rating: 5 }
    { _id: "R902", rating: 4 }
    { _id: "R903", rating: 5 }
  ]
}
```



max 3
predictable size

VS

UNBOUNDED ARRAY

 training_store • products

```
{
  _id: "P204",
  name: "MongoDB Basics",
  reviews: [
    { _id: "R901", rating: 5 }
    { _id: "R902", rating: 4 }
    { _id: "R903", rating: 5 }
    ⋮
    { _id: "R10,000+", rating: 4 }
  ]
}
```

keeps growing



large document



costly updates

Bounded

A customer's few addresses; an order's handful of line items.

Unbounded

Every review a product will ever get — it keeps growing.

Hard limit

One BSON document can be at most 16 MiB.

TRAINING_STORE Reviews live in their own collection.

KEY TAKEAWAY Ask how large every array can become — embed the bounded ones, move the unbounded ones out.

EXERCISE: CLASSIFY THE TRAINING_STORE RELATIONSHIPS

Name the cardinality and the growth before you pick a model.

PRACTICE EXERCISE

10 MIN

SCENARIO

Five relationships from the dataset. For each, decide how many sit on the many side and whether it grows without limit.

- 1 customer → addresses
- 2 order → line items
- 3 customer → orders
- 4 product → reviews
- 5 orders ↔ products

YOUR TASKS

- 1 Name each relationship's type
- 2 Say bounded or unbounded, and why
- 3 Decide: embed or reference
- 4 Find the field that links each pair

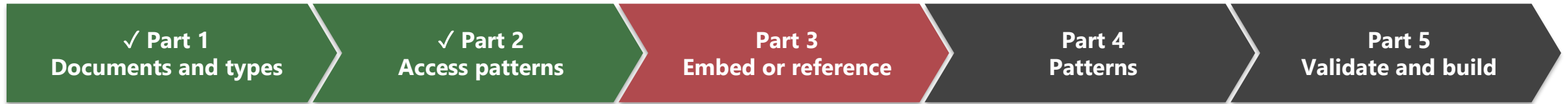
EXPECTED OUTCOME

- ✓ 1, 2: one-to-few, bounded — embed
- ✓ 3: one-to-many, grows — customerId
- ✓ 4: grows without limit — productId
- ✓ 5: many-to-many — items[].productId
- ✓ Growth, not nesting, decides

KEY TAKEAWAY Cardinality and growth decide the model — the same question works for any relationship.

PART 3: EMBEDDING VERSUS REFERENCING

The two ways MongoDB represents a relationship.



SO FAR

- ✓ Design starts from access patterns
- ✓ Name the relationship and count the many side
- ✓ Arrays must stay bounded

NEXT QUESTION

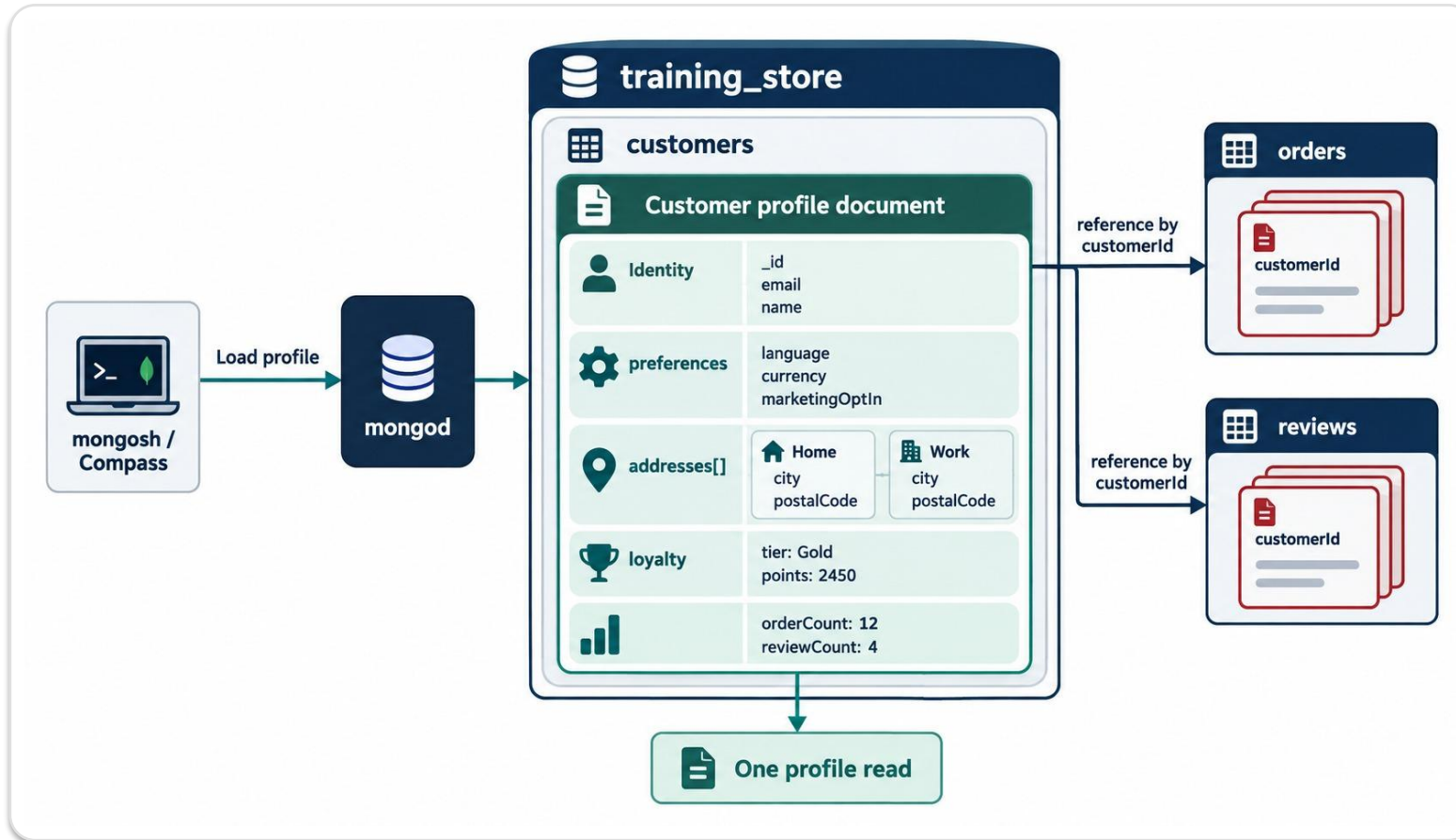
When should related data live inside the document — and when in another collection?

IN THIS PART

- 1 Embedding: customer profiles
- 2 Referencing: product reviews
- 3 How to choose between them
- 4 A decision tree
- 5 Orders: a hybrid model
- 6 Historical snapshots

EMBEDDING: CUSTOMER PROFILES

Addresses belong to the customer and are read with the profile.



customers: C101 (trimmed)

```
{ customerNumber: "C101",  
  name: { first: "Aisha",  
          last: "Khan" },  
  addresses: [ {  
    type: "SHIPPING",  
    city: "Toronto", ... } ] }
```

Advantages

One read, no join, atomic updates inside one document.

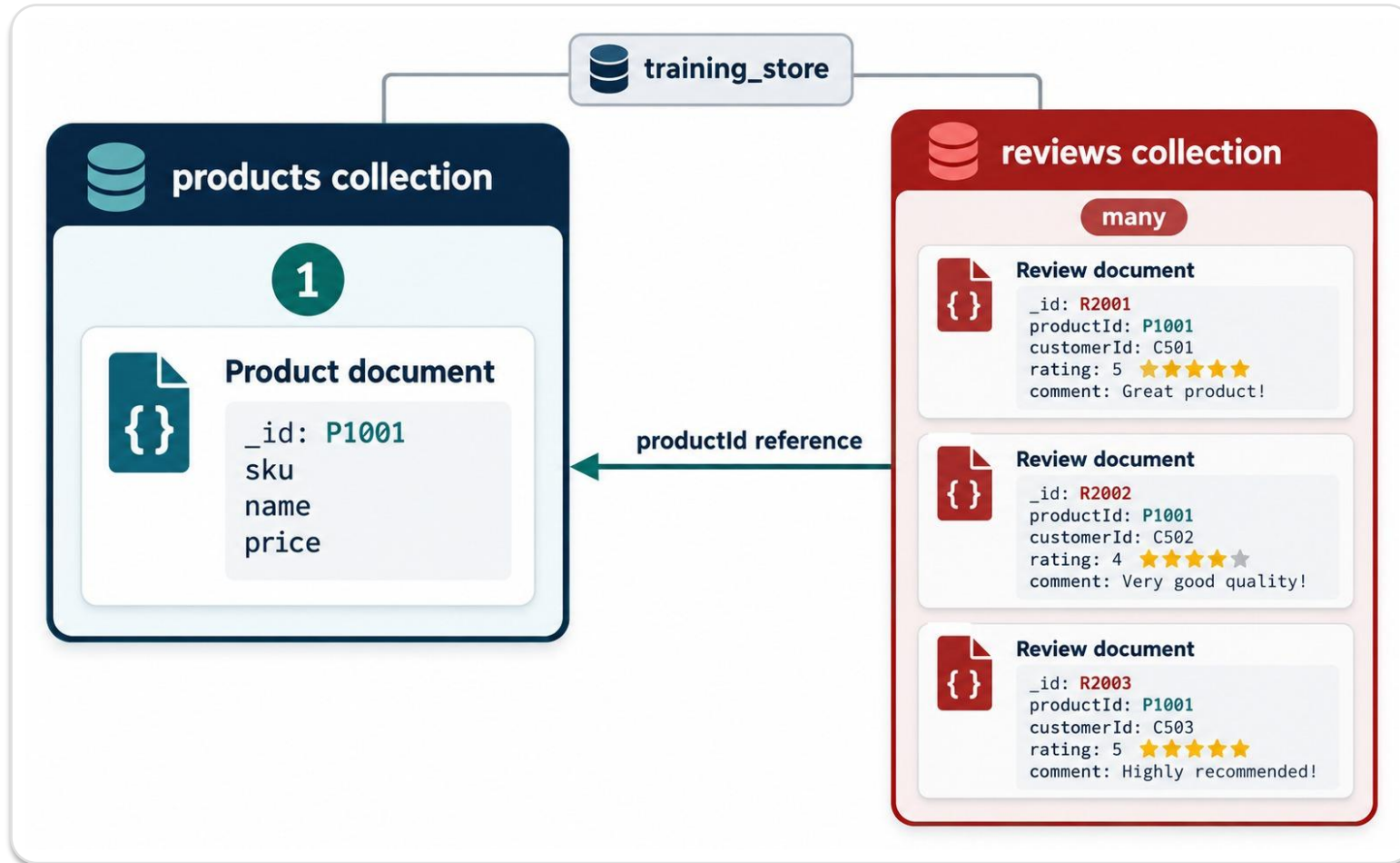
Limitations

Duplication, bigger documents — and never for unbounded data.

KEY TAKEAWAY Embed data that belongs to the parent, stays small and is read with it.

REFERENCING: PRODUCT REVIEWS

Each review points to its product and its author.



reviews: the L100 review (trimmed)

```
{ productId: ObjectId("..."),
  sku: "L100",
  customerId: ObjectId("..."),
  rating: 5,
  title: "Excellent for travel" }
```

Advantages

Small documents, independent growth, no large duplication.

Limitations

A second query or \$lookup; multi-document changes may need a transaction.

KEY TAKEAWAY Reference data that is shared, changes on its own or grows without limit.

HOW TO CHOOSE BETWEEN THEM

Nine considerations from the modeling guide.

Consideration	Embed	Reference
Normally read together	Yes	Possibly
Small and bounded	Yes	Not required
Belongs to one parent	Yes	Possibly
Grows continuously	No	Yes
Shared by many records	No	Yes
Changes independently	Sometimes	Yes
One-query retrieval matters	Yes	Needs \$lookup
Duplication would be large	No	Yes
Atomic update required	Strong choice	Transaction

Rule

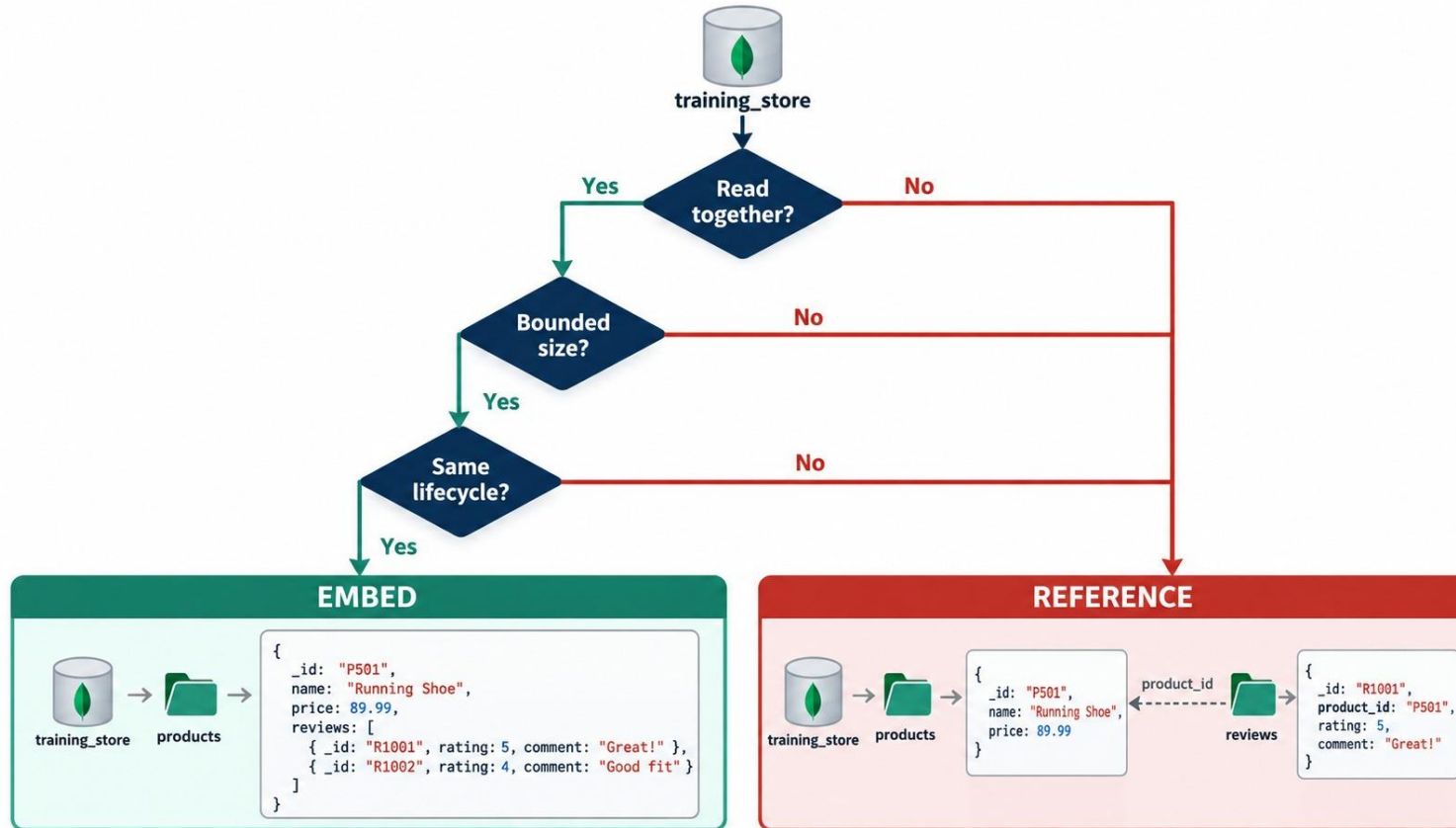
Embed bounded data that belongs to the parent; reference independent, shared or unbounded data.

EXERCISE 3.3 Eight relationships to decide.

KEY TAKEAWAY Embed bounded data that belongs to the parent; reference independent, shared or unbounded data.

AN EMBEDDING DECISION TREE

Three questions — all yes means embed.



Read together?

Is it shown with the parent on the common page?

Bounded size?

Does it have a small, known maximum?

Same lifecycle?

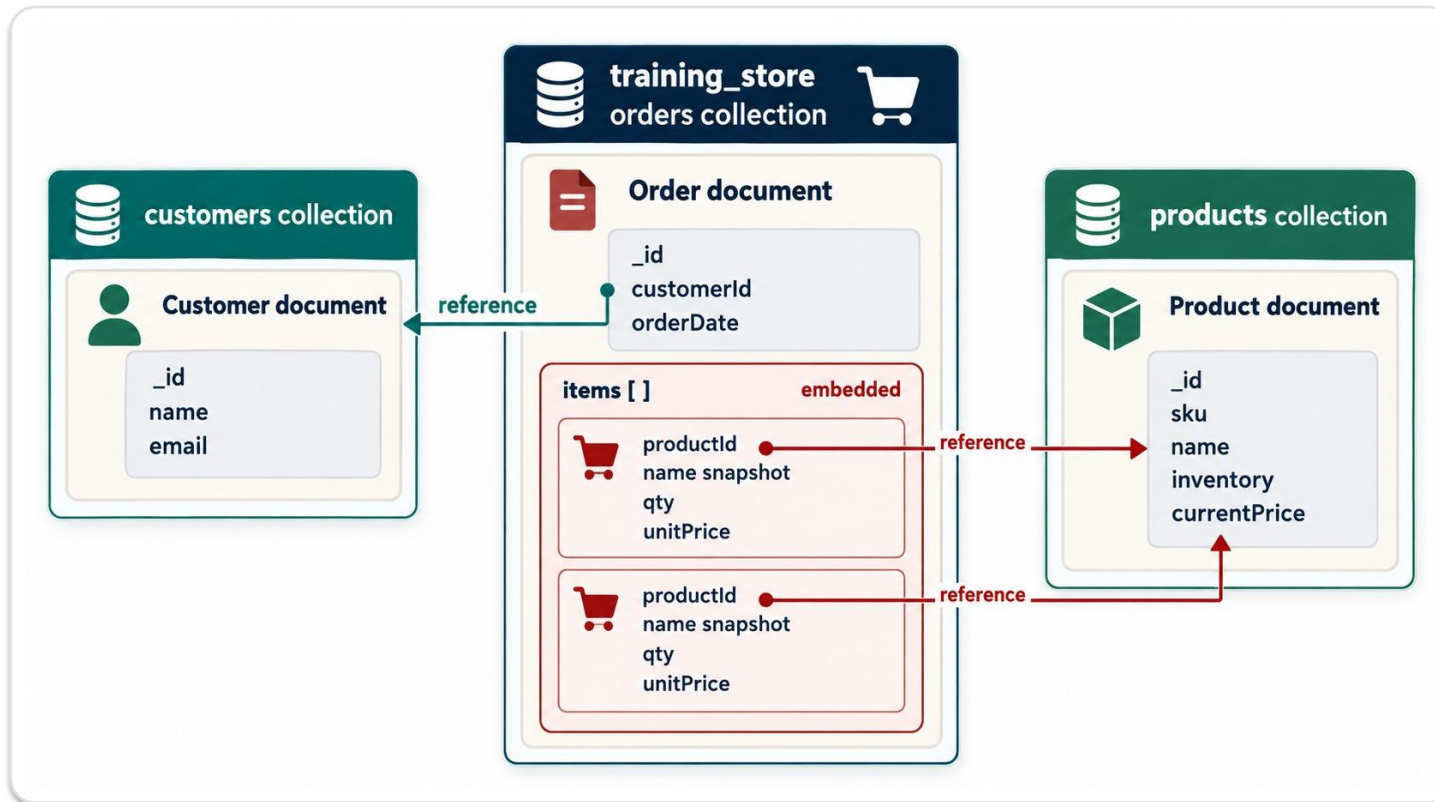
Is it created and deleted with the parent?

REVIEWS Fail bounded size — so they are referenced.

KEY TAKEAWAY Embed only when the data is read together, bounded and shares the parent's lifecycle.

MODELING ORDERS: A HYBRID APPROACH

Reference the customer and products; embed items and the address.



orders: O5001 (trimmed)

```
{
  orderNumber: "05001",
  customerId: ObjectId("..."),
  items: [
    {
      productId: ObjectId("..."),
      sku: "L100",
      name: "Business Laptop",
      quantity: 1,
      unitPrice: Decimal128("1299.99")
    },
    // A410 x 2
  ],
  shippingAddress: {
    city: "Toronto",
    ...
  },
  total: Decimal128("1514.17")
}
```

Reference

`customerId` → the customer; `productId` → the catalog.

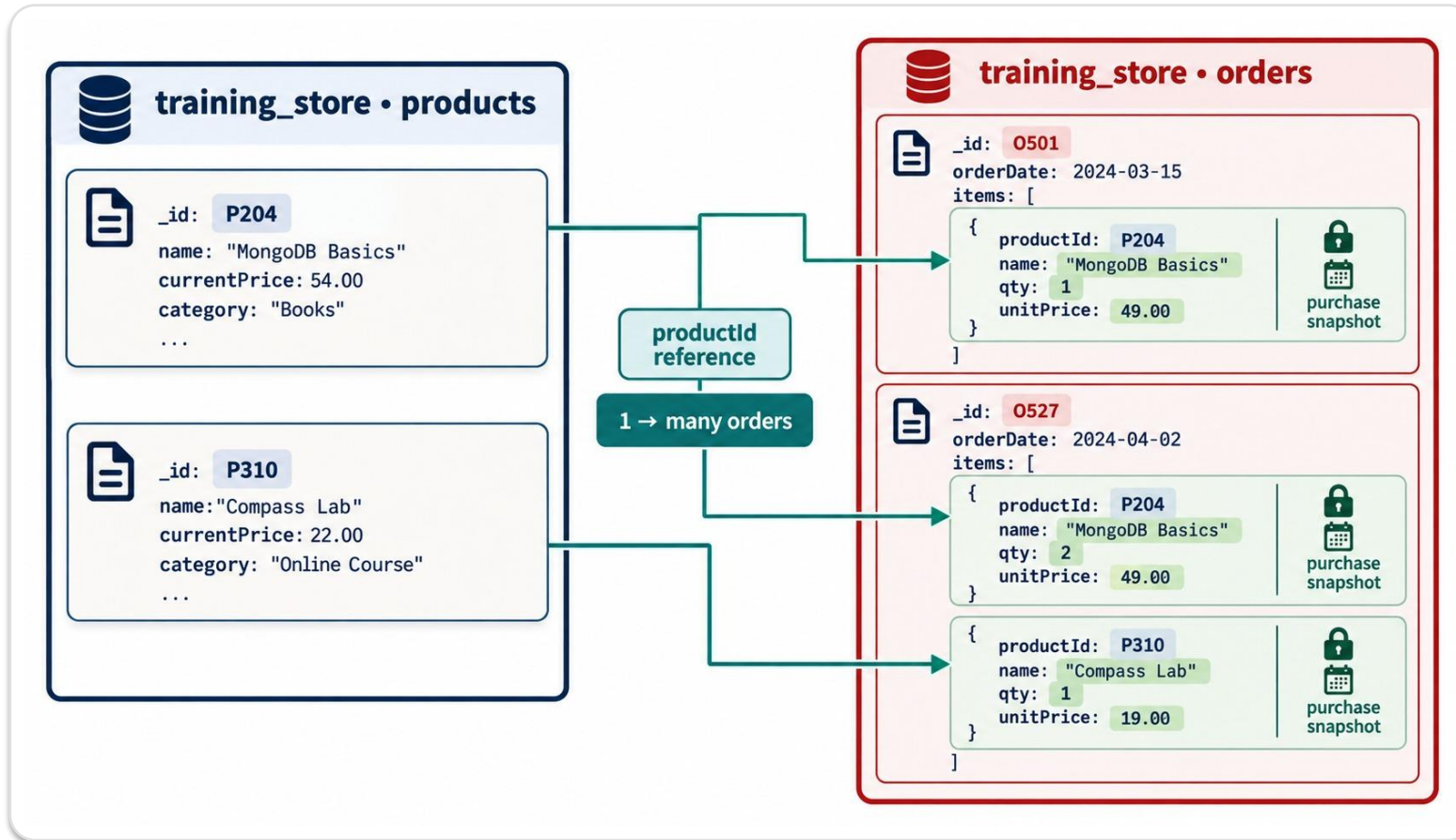
Embed

Line items and the shipping address, read with the order.

KEY TAKEAWAY One order document answers "show this order" — while customers and products keep changing on their own.

HISTORICAL SNAPSHOTS

The order keeps what the customer paid, not today's price.



Snapshot

unitPrice and name are copied at purchase — and never synced.

Current price

Lives only in products; the catalog can change freely.

In training_store

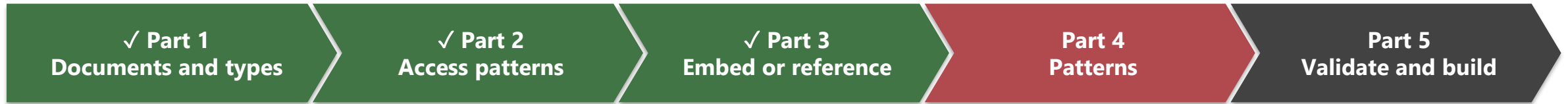
If L100's price changes, O5001 still shows 1299.99.

COPY WITH CARE Freeze history — don't copy data that must stay current.

KEY TAKEAWAY Duplication is deliberate when it preserves history — a snapshot should never follow the catalog.

PART 4: ANTI-PATTERNS AND SCHEMA PATTERNS

What goes wrong — and the proven fixes.



SO FAR

- ✓ Embed bounded data that belongs to the parent
- ✓ Reference shared, independent or unbounded data
- ✓ Snapshots preserve history on purpose

NEXT QUESTION

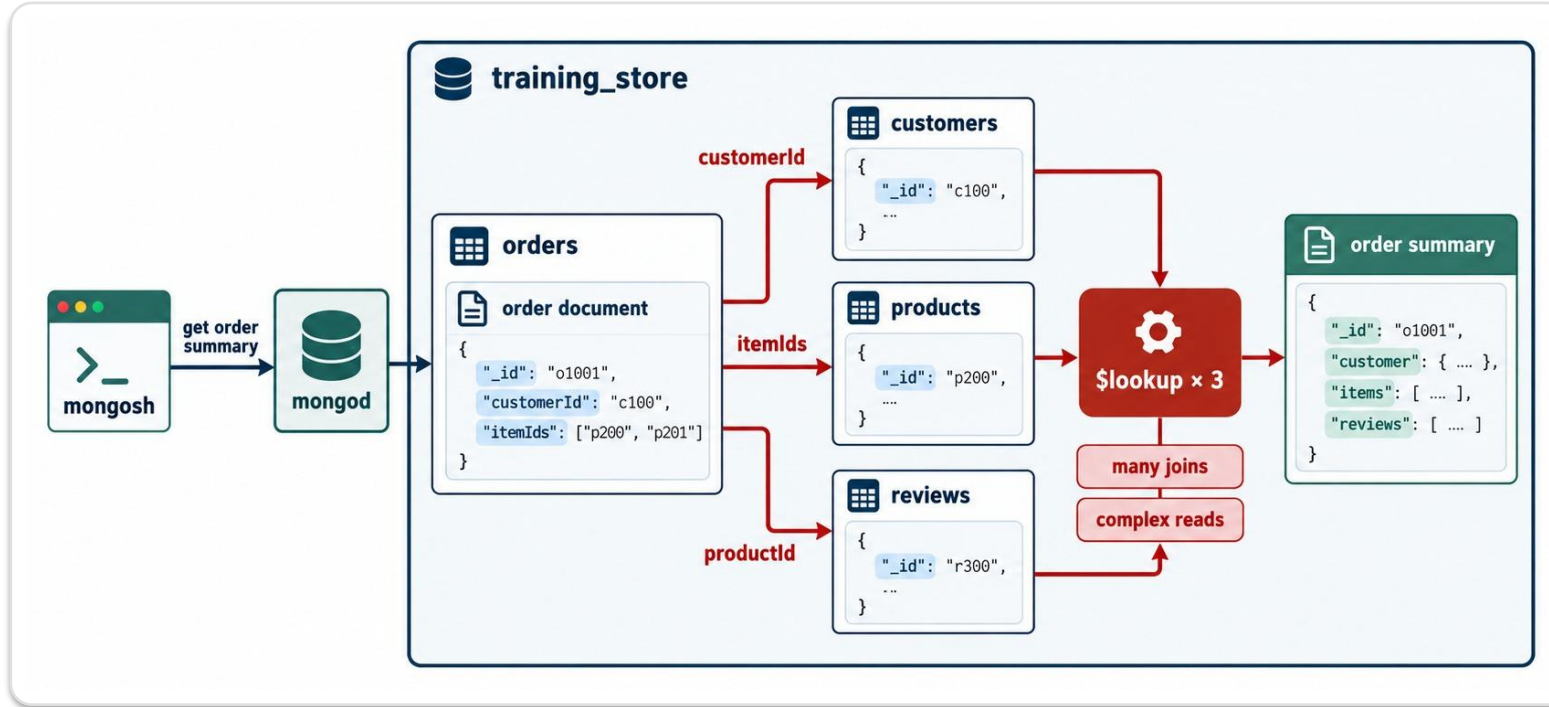
Which mistakes do first designs make — and which patterns solve common modeling problems?

IN THIS PART

- 1 Collections copied from tables
- 2 Inconsistent names and types
- 3 Seven schema patterns
- 4 Attribute, Subset and Computed up close

ANTI-PATTERN: COLLECTIONS COPIED FROM TABLES

One collection per table turns every page into joins.



The mistake

A collection for every small component; the app rebuilds each order.

The cost

$\$lookup \times 3$ — more round trips, more code, slower pages.

The fix

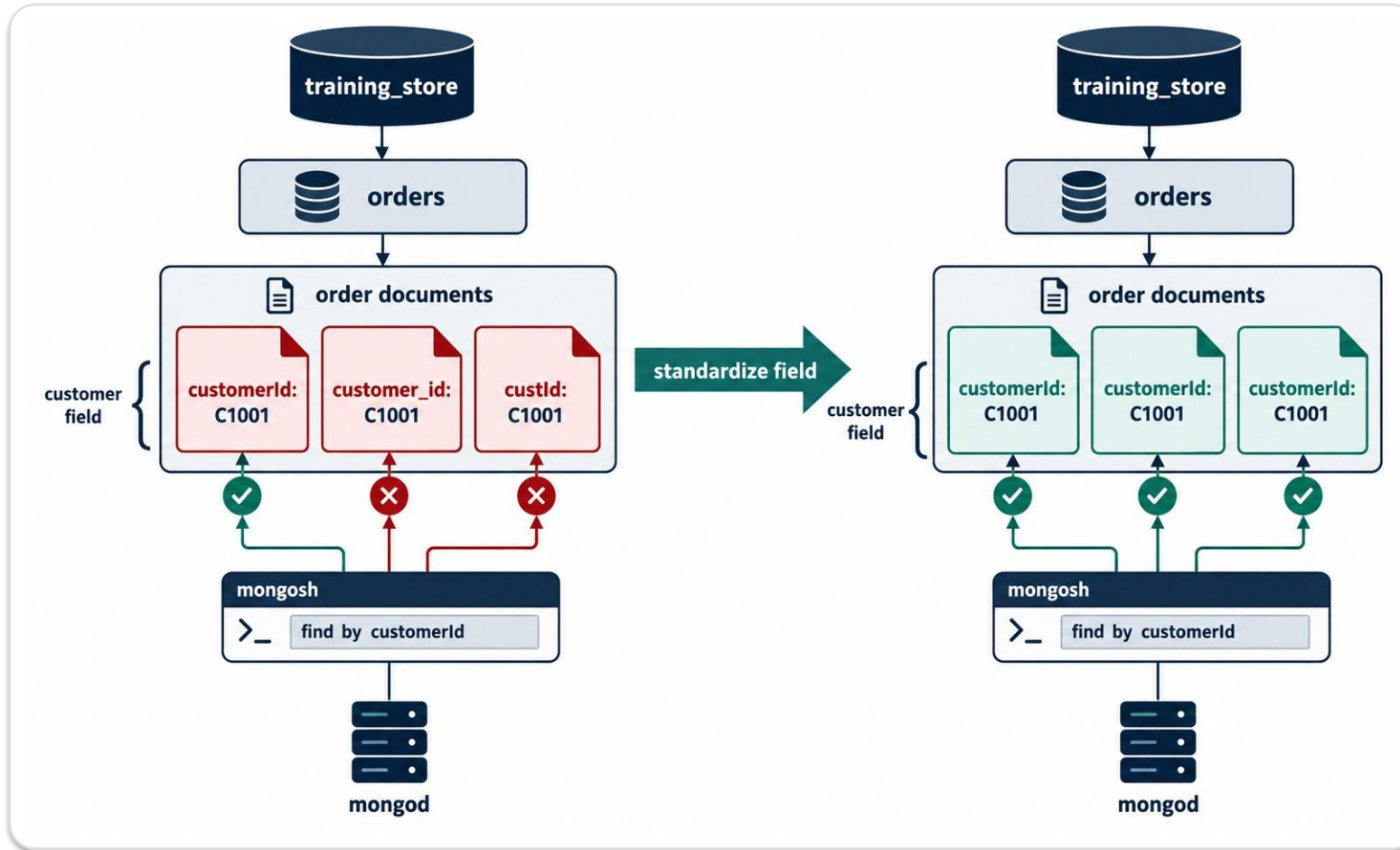
Embed what the page reads together; reference the rest.

TRAINING_STORE One O5001 document is one read.

KEY TAKEAWAY Don't translate tables into collections — model the documents the application reads.

ANTI-PATTERN: INCONSISTENT NAMES AND TYPES

One field should have one name — and one type.



Names

customerId, customer_id, custId — a query on one misses the others.

✗ Types: three shapes, one field

```
{ price: 29.99 }  
{ price: "29.99" }  
{ price: { amount: 29.99 } }
```

The fix

One name, one structure per field — then enforce it with validation.

TRAINING_STORE camelCase everywhere:
customerId, createdAt.

KEY TAKEAWAY Consistency is a correctness rule: queries match exact names and compare by type.

SEVEN COMMON SCHEMA PATTERNS

Named solutions to recurring modeling problems.

Pattern	Use it when	Example
Attribute	Many optional, searchable fields	Product specifications
Bucket	Many small time-based events	Sensor readings per hour
Subset	The page needs only the hot few	Latest three reviews
Computed	The same calculation repeats	Stored average rating
Extended reference	A few fields read with the link	Customer name on an order
Outlier	A few documents are far larger	A viral product's reviews
Polymorphic	Different shapes, shared queries	Laptops, shoes, books

Every pattern costs

Faster reads usually mean extra writes or extra code.

Already here

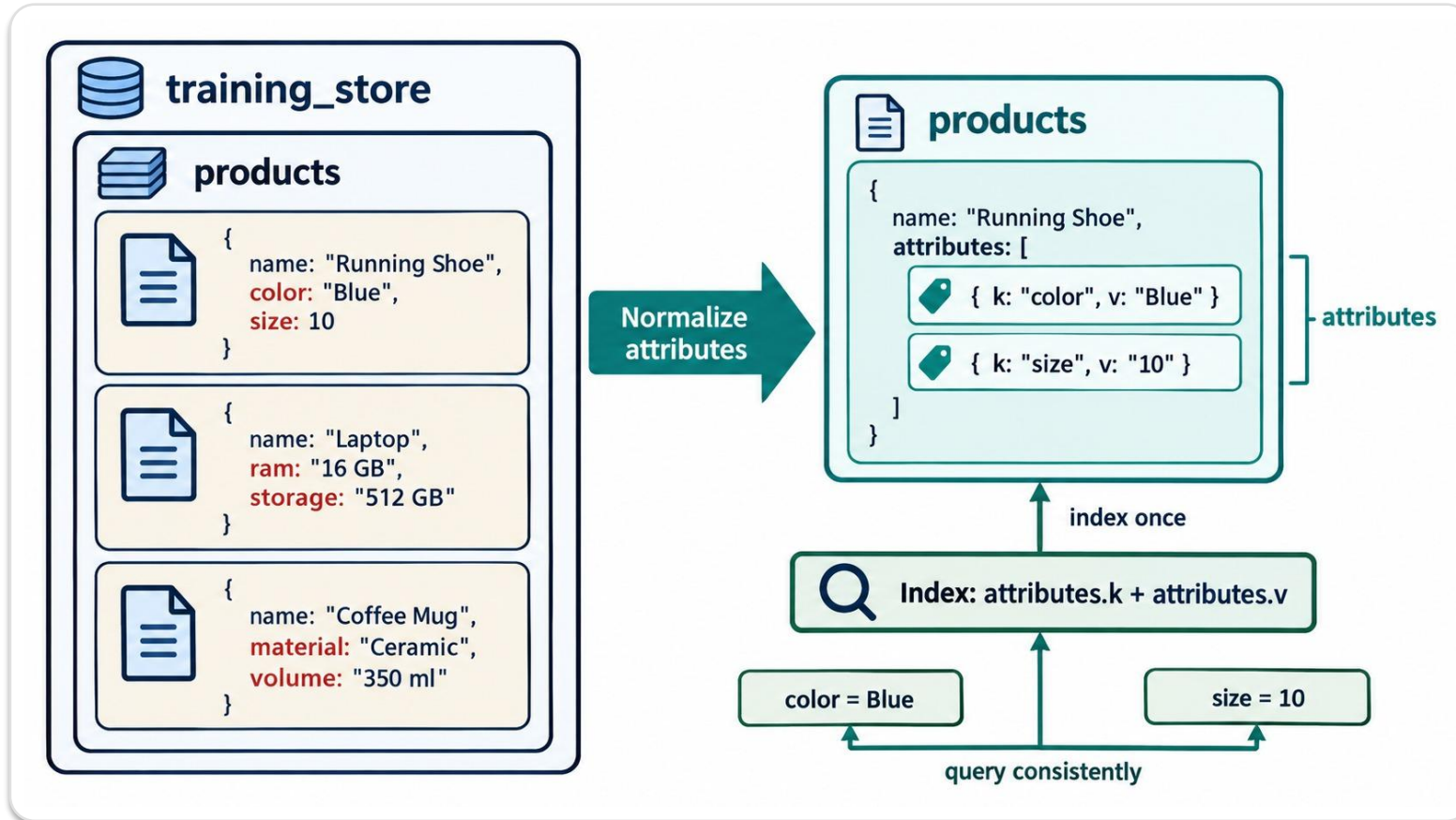
products is polymorphic; order items use extended references.

EXERCISE 3.7 Match seven requirements to these patterns.

KEY TAKEAWAY Patterns are proven trade-offs — pick one only when its problem is really in your access patterns.

THE ATTRIBUTE PATTERN

Many optional attributes, one index.



The problem

Every searchable attribute would need its own index.

The pattern

Store `{ k, v }` pairs; index `attributes.k` and `attributes.v` together.

training_store today

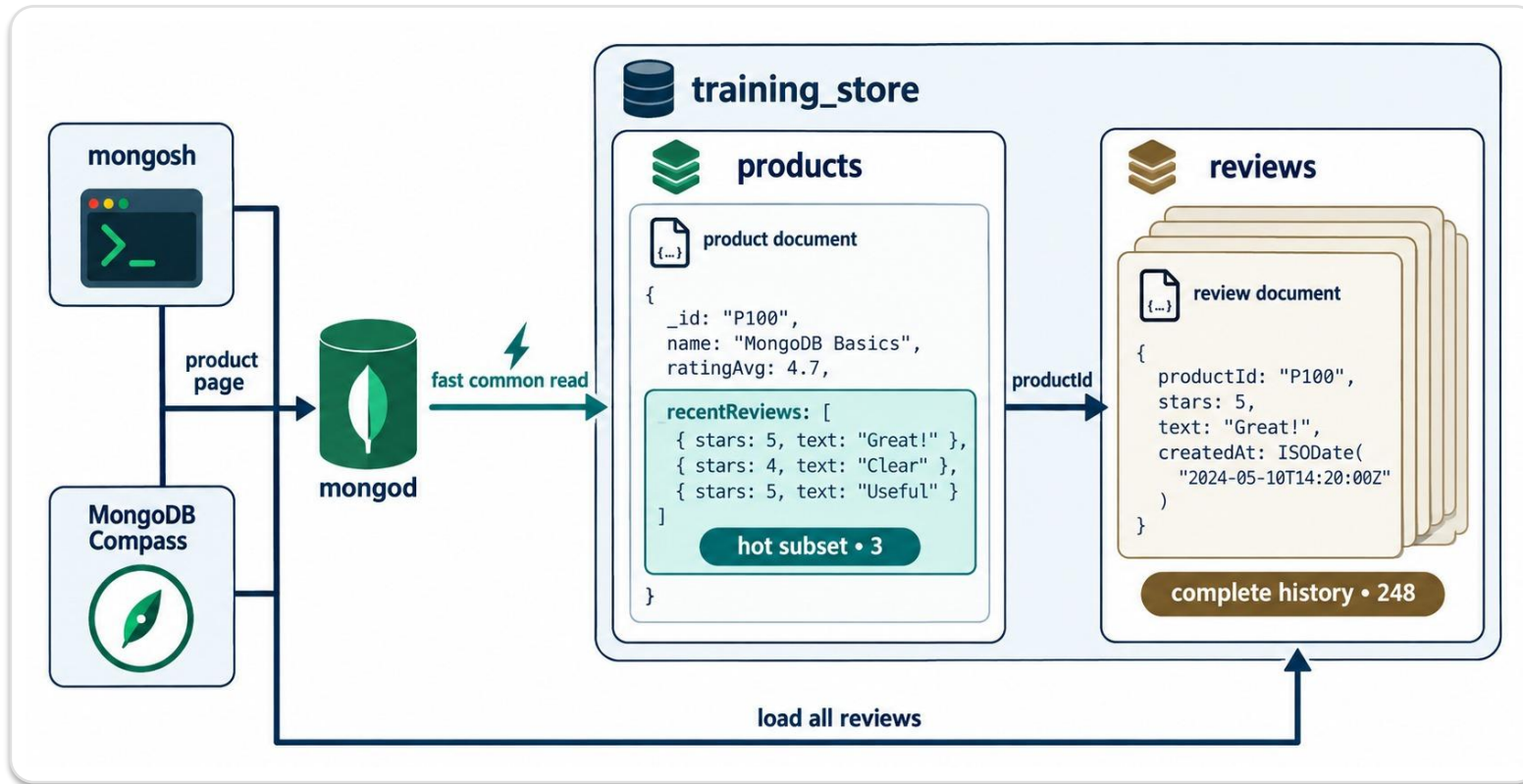
`attributes` is a plain object — fine for a few known fields.

TRADE-OFF Simpler indexing, less natural documents.

KEY TAKEAWAY Use the Attribute pattern when many optional fields must all be searchable.

THE SUBSET PATTERN

Embed the hot few; keep the full history separate.



The problem

The product page needs three reviews, not all 248.

The pattern

Embed a small recentReviews array; keep every review in reviews.

Trade-off

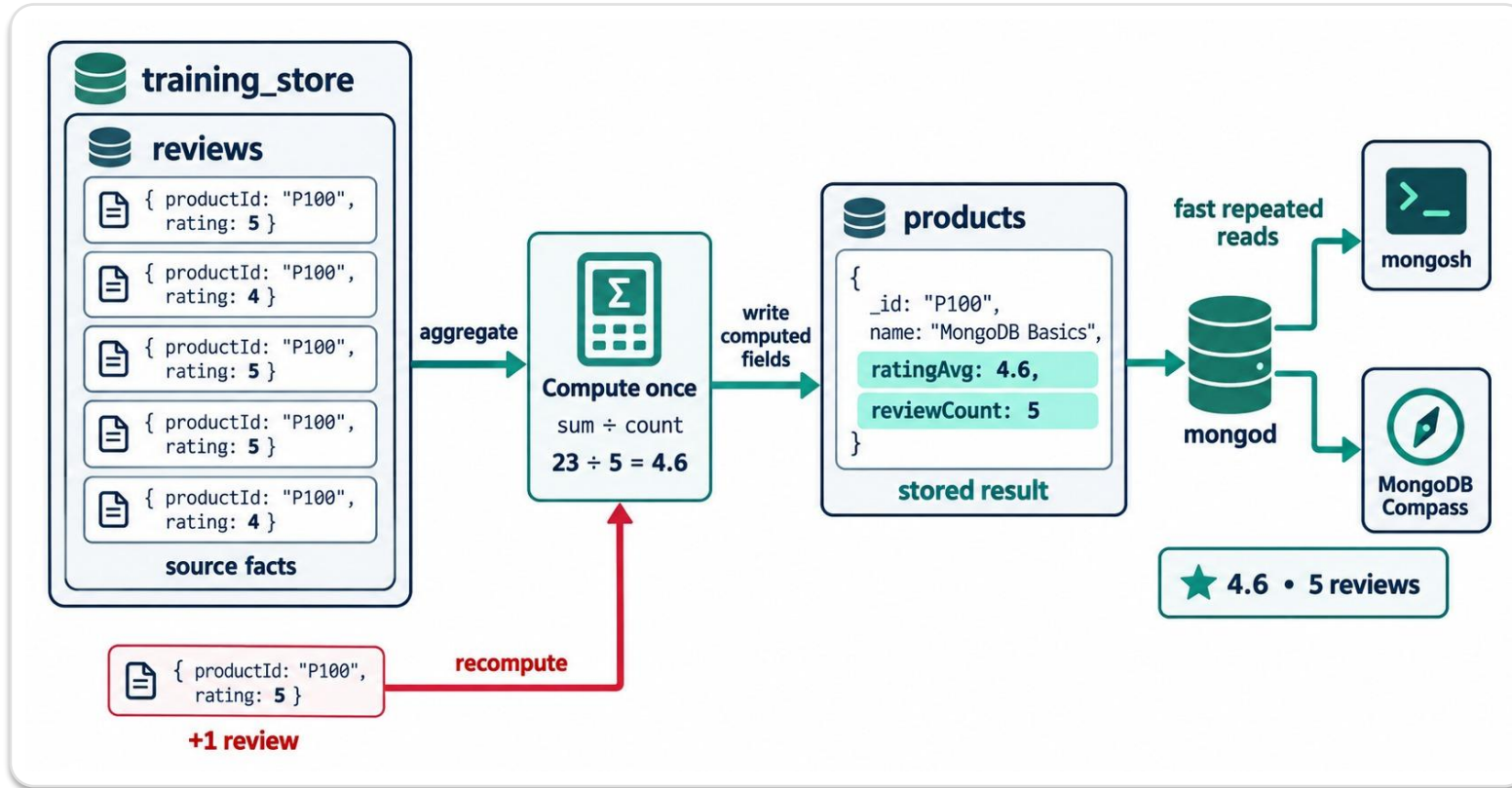
A new review is written in two places.

TRAINING_STORE Not used yet — reviews live only in reviews.

KEY TAKEAWAY Subset keeps the common page to one read without embedding an unbounded array.

THE COMPUTED PATTERN

Calculate once, store the result, read it many times.



The problem

Averaging every review on every page view.

The pattern

Store `ratingAvg` and `reviewCount` on the product.

Trade-off

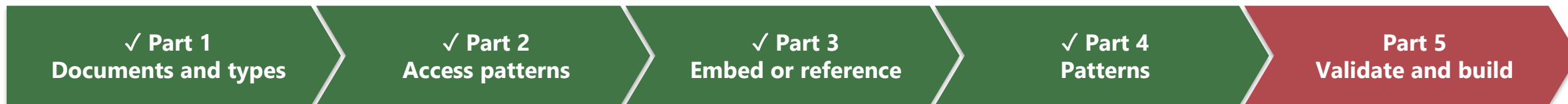
Recompute on each new review, or on a schedule.

MODULE 5 You'll compute averages with `$group`.

KEY TAKEAWAY Store a calculated value when it is read far more often than its inputs change.

PART 5: VALIDATE, EVOLVE AND BUILD

Enforce the model, plan for change, then build it.



SO FAR

- ✓ Avoid table-shaped collections and mixed types
- ✓ Seven patterns solve recurring problems
- ✓ Every pattern is a trade-off

NEXT QUESTION

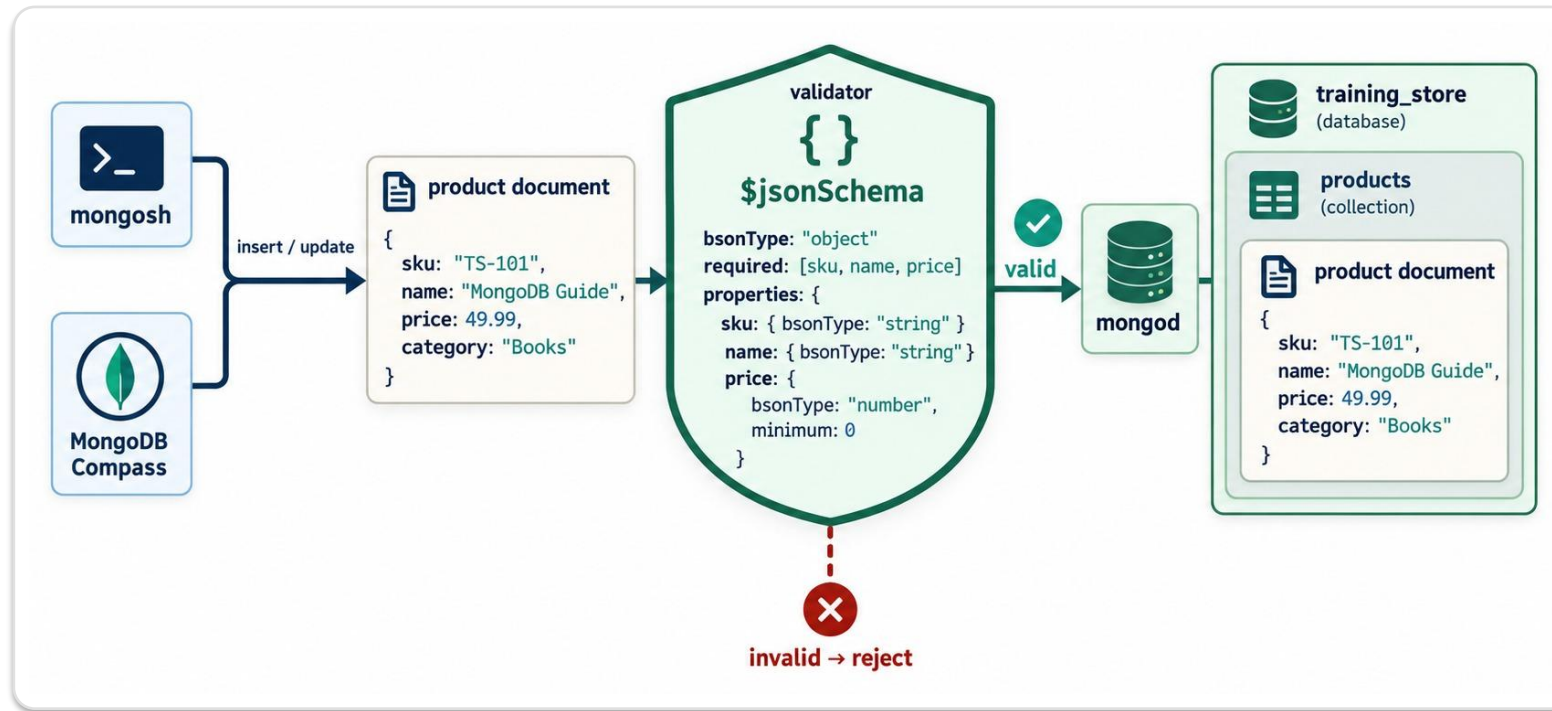
How do we enforce the model, change it safely — and build it in `training_store`?

IN THIS PART

- 1 \$jsonSchema validation
- 2 Validation level and action
- 3 Evolving a schema
- 4 Querying nested fields and arrays
- 5 Joining references with \$lookup
- 6 Indexes for the access patterns

SCHEMA VALIDATION WITH \$JSONSCHEMA

Flexible fields, enforced rules.



required

Fields every document must have.

bsonType

The type each field must hold: string, decimal, bool, date.

minimum

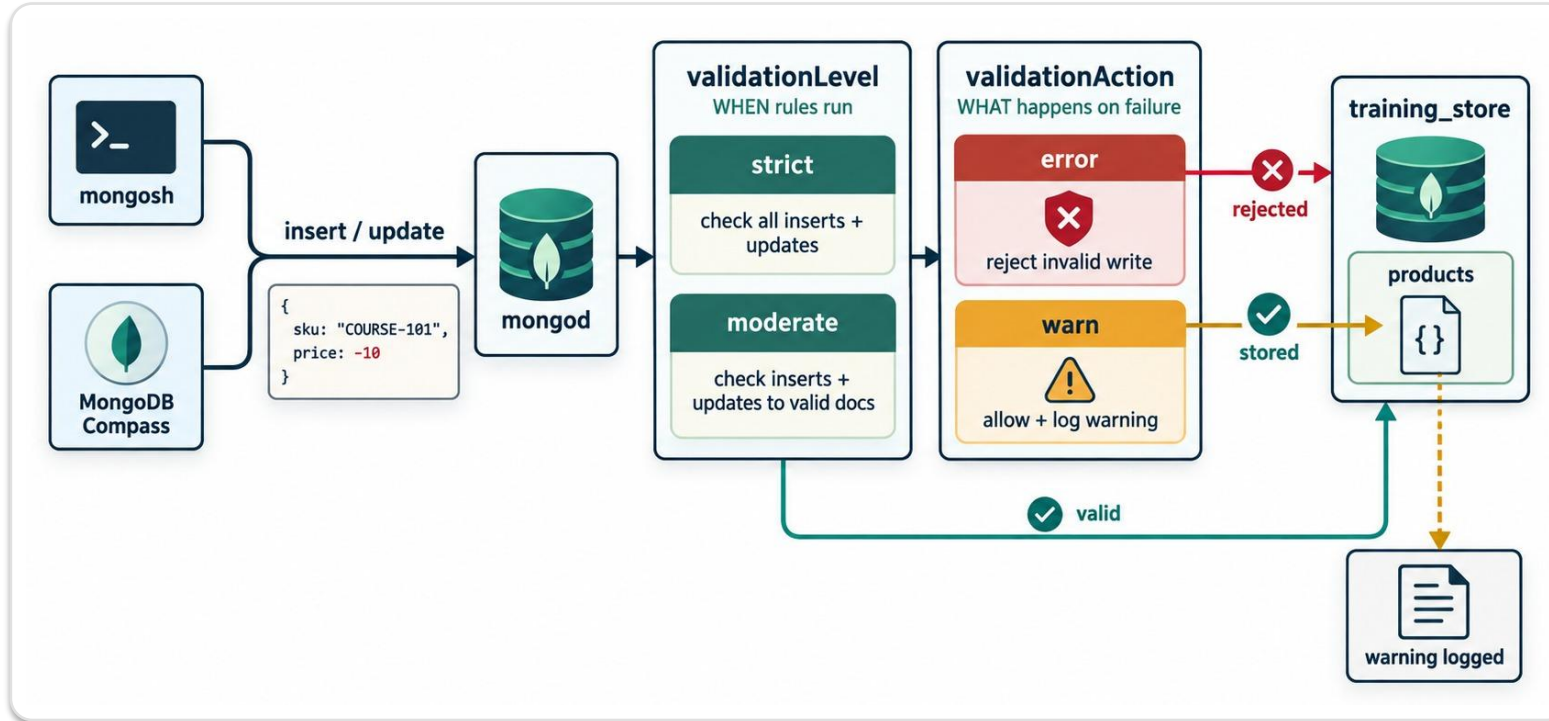
Range rules — price can't be negative.

LAB 2, STEP 6 Requires decimal for price — stricter than number.

KEY TAKEAWAY Validation lets the database reject documents that break the model's contract.

VALIDATION LEVEL AND ACTION

When rules run — and what a failure does.



validationLevel

strict checks every write; moderate skips already-invalid documents.

validationAction

error rejects the write; warn stores it and logs a warning.

Relax rules on old data

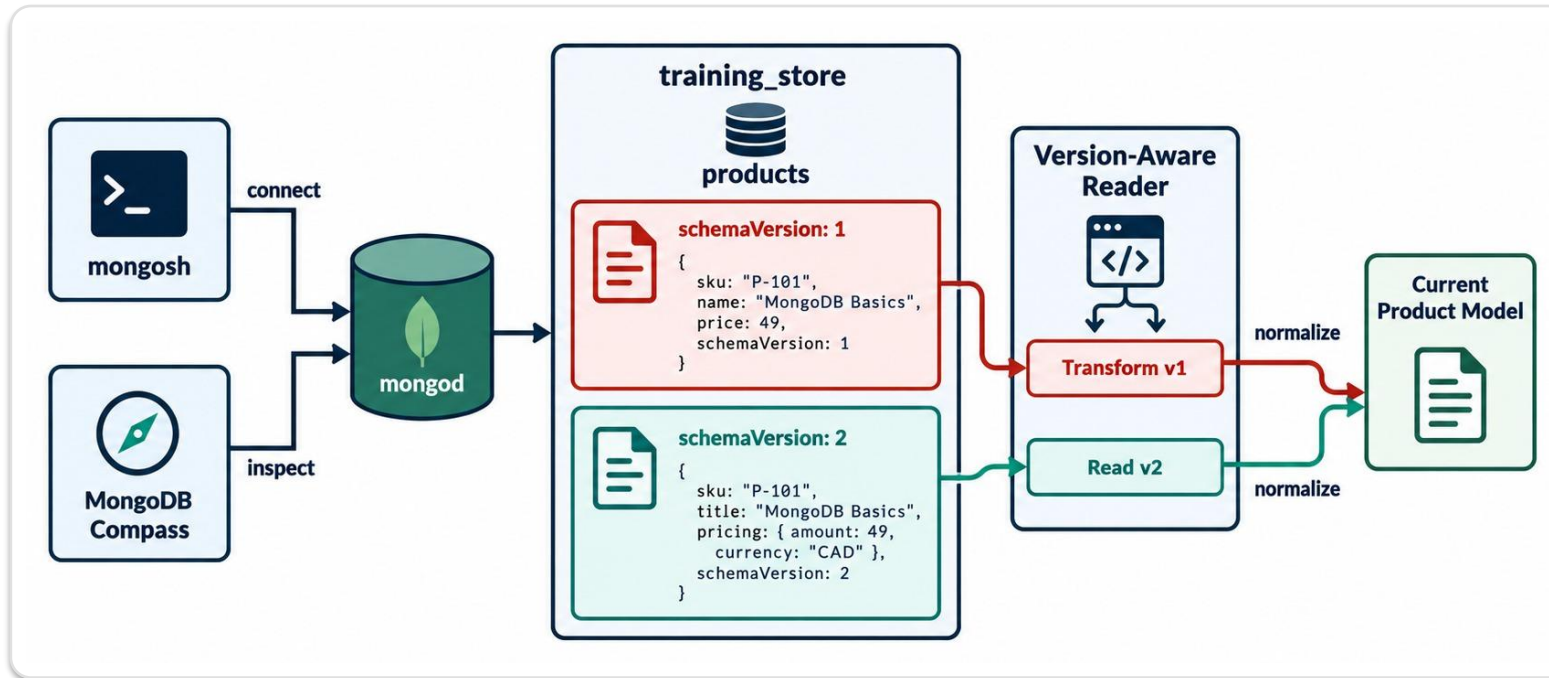
```
db.runCommand({
  collMod: "validated_products",
  validationLevel: "moderate",
  validationAction: "warn" })
```

DEFAULT strict + error.

KEY TAKEAWAY Keep strict and error for new data; use moderate or warn only while old documents are fixed.

EVOLVING A SCHEMA SAFELY

Old and new shapes can coexist while you migrate.



Non-breaking first

Adding an optional field keeps old readers working.

Version the shape

schemaVersion tells the reader which shape it has.

Migrate in steps

Tolerant readers → new writes → backfill → require.

DEMO 3.6 v1 price becomes v2 pricing: { amount, currency }.

KEY TAKEAWAY Evolve a schema in steps — readers first, backfill later, validation last.

QUERY NESTED FIELDS AND ARRAYS

Lab 2, Step 5: dot notation walks into documents and arrays.

Products

```
db.products.find({ category: "LAPTOP" })
db.products.find(
  { "attributes.memoryGB": 16 })
db.products.find({ tags: "technology" })
```

Customers and orders

```
db.customers.find(
  { "addresses.city": "Toronto" })
db.orders.find(
  { "items.sku": "L100" })
```

Hand-built core

L100 · L100 · B300 · C101 · O5001.

After load.js

LAPTOP returns L100, L110, L190; items.sku finds four orders.

Arrays

tags: "technology" matches any element of the array.

KEY TAKEAWAY Dot notation reaches into embedded documents and arrays of documents — always in quotes.

JOIN REFERENCED DATA WITH \$LOOKUP

The cost of referencing: combining needs another query or a join.

Reviews for L100, with the product name

```
const l100 = db.products.findOne({ sku: "L100" })
db.reviews.aggregate([
  { $match: { productId: l100._id } },
  { $lookup: { from: "products",
    localField: "productId",
    foreignField: "_id", as: "product" } },
  { $unwind: "$product" },
  { $project: { _id: 0, rating: 1, title: 1,
    productName: "$product.name" } }
])
```

✓ Result (full load)

```
[
  {
    rating: 5,
    title: 'Excellent for travel',
    productName: 'Business Laptop'
  }
]
```

\$lookup

Joins each review to the product whose `_id` equals its `productId`.

Needs the full load

The hand-built core has no reviews yet.

Shortcut

Reviews copy `sku`, so `find({ sku: "L100" })` needs no join.

KEY TAKEAWAY Referenced data stays independent — combining it costs a `$lookup` or a second query.

INDEXES THAT FOLLOW THE ACCESS PATTERNS

Design them now — Module 6 creates and tests them.

✓ Already created by load.js

```
db.products.getIndexes()  
// _id_, sku_unique  
db.customers.getIndexes()  
// _id_, customerNumber_unique,  
// email_unique on contact.email  
db.orders.getIndexes()  
// _id_, orderNumber_unique
```

Planned from the access patterns

```
// browse by category and price  
{ category: 1, price: 1 }      // products  
// a customer's order history  
{ customerId: 1, createdAt: -1 } // orders  
// a product's reviews, newest first  
{ productId: 1, createdAt: -1 } // reviews
```

Unique keys

sku, customerNumber, orderNumber and email can't repeat.

Pattern → index

Each planned index serves one row of the access table.

Module 6

explain() will show each index being used.

KEY TAKEAWAY Every important access pattern should have an index to support it — plan them with the model.

EXERCISE: PREDICT THE NESTED QUERY RESULTS

Read each query and say what it returns — before you run it.

PRACTICE EXERCISE

10 MIN

SCENARIO

Run after load.js. Use the dataset facts: shoe sizes, accessory connections, customer countries and order items.

```
db.products.find(
  { "attributes.sizes": 11 })
db.products.find(
  { "attributes.connection": "USB-C" })
db.customers.find(
  { "addresses.country": "USA" })
db.orders.countDocuments(
  { "items.sku": "A410" })
```

YOUR TASKS

- 1 Predict each query's result
- 2 Name the embedded path it follows
- 3 Say where an array is crossed
- 4 Check on the instructor's screen

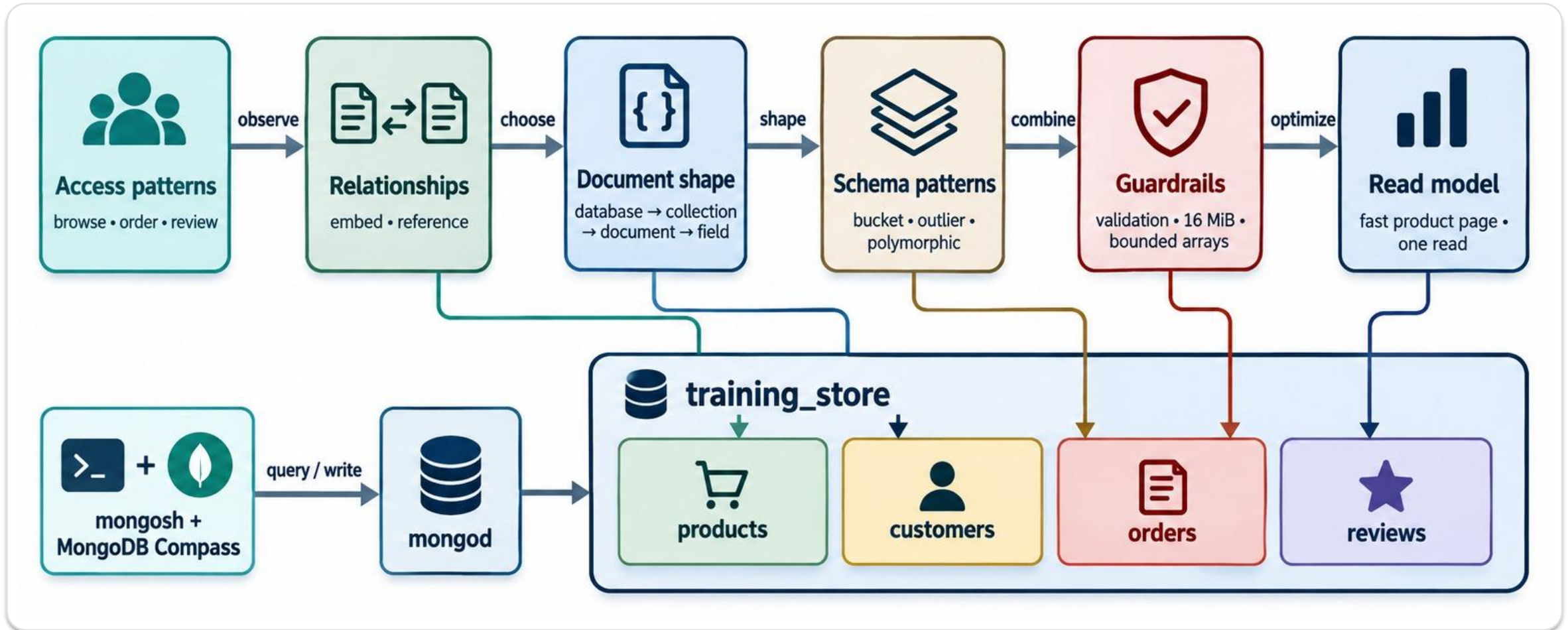
EXPECTED OUTCOME

- ✓ sizes 11 → Trail Shoe (S210)
- ✓ USB-C → USB Hub, Docking Station
- ✓ USA → Luis (C204), Maya (C620)
- ✓ A410 → 3 orders
- ✓ Arrays: sizes, addresses, items

KEY TAKEAWAY Dot notation follows the document's shape — arrays and embedded documents alike.

MODULE 3 AT A GLANCE

From access patterns to a read model, on one page.



KEY TAKEAWAY Model from the access patterns, embed what is bounded and owned, reference the rest — then enforce it.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1

What is the difference between a collection and a document?

2

What does BSON add to JSON?

3

What three things does `_id` guarantee?

4

When is embedding appropriate?

5

When is referencing appropriate?

6

Why are unbounded arrays dangerous?

7

What is the maximum size of one BSON document?

8

Why does an order copy the product price?

9

What does schema validation enforce?

10

Which pattern stores a calculated result?

EXERCISE 3.1 — IDENTIFY DOCUMENT COMPONENTS

Module 3 checkpoint A: label every part of a customer document.

OFFICIAL CHECKPOINT A

10 MIN

labs/day-01/exercises/exercise-3.1-identify-document-components.md

SCENARIO

Pairs. Label the sample customer, then list candidate required fields.

```
{ _id: ObjectId("..."),
  customerNumber: "C101",
  name: { first: "Aisha",
          last: "Khan" },
  email: "aisha@example.com",
  tags: ["premium", "newsletter"],
  active: true,
  createdAt: ISODate("...") }
```

YOUR TASKS

- 1 Mark `_id` and say what it guarantees
- 2 Mark the scalars and the embedded name
- 3 Label the array, Boolean and date
- 4 List candidate required fields

SOLUTION (AFTER DEBRIEF)

- ✓ `_id`: unique in the collection
- ✓ Scalars: `customerNumber`, `email`
- ✓ `name`: embedded, `first` and `last`
- ✓ `tags` array · `active` · `createdAt`
- ✓ Required: `_id`, `customerNumber`, `name`, `email`, `active`, `createdAt`

KEY TAKEAWAY Naming each part of a document is the first step to designing one.

EXERCISE 3.2 — DISCOVER APPLICATION ACCESS PATTERNS

Module 3 checkpoint B: turn requirements into access patterns.

OFFICIAL CHECKPOINT B

15 MIN

labs/day-01/exercises/exercise-3.2-discover-access-patterns.md

SCENARIO

The store must support product search, product pages, customer profiles, order placement, order history, reviews and inventory display.

YOUR TASKS

- 1 Name the three most frequent reads
- 2 Write the data each returns together
- 3 Fill four or more table rows
- 4 Mark embed, reference or snapshot

SOLUTION (AFTER DEBRIEF)

- ✓ Product by SKU · order by number · profile
- ✓ Returned data is specific, not everything
- ✓ Add order history and paginated reviews
- ✓ Purchase-time price is a snapshot
- ✓ At least one embed and one reference

KEY TAKEAWAY Access patterns turn vague requirements into concrete document shapes.

EXERCISE 3.3 — EMBED OR REFERENCE?

Module 3 checkpoint C: decide eight relationships and justify each.

OFFICIAL CHECKPOINT C

15 MIN

labs/day-01/exercises/exercise-3.3-embed-or-reference.md

SCENARIO

Write Embed or Reference and a one-line reason: ownership, size or growth.

- 1 Primary contact information
- 2 Complete order history
- 3 Order line items
- 4 A product's millions of reviews
- 5 An employee's office address
- 6 Blog post tags
- 7 Purchase-time product price
- 8 Complete transaction history

YOUR TASKS

- 1 Decide relationships 1–4
- 2 Decide relationships 5–8
- 3 Justify with bounded or unbounded
- 4 Explain the one 'it depends'

SOLUTION (AFTER DEBRIEF)

- ✓ 1 Embed · 2 Reference
- ✓ 3 Embed · 4 Reference
- ✓ 5 Depends on reuse
- ✓ 6 Embed · 7 Embed (snapshot)
- ✓ 8 Reference

KEY TAKEAWAY Unbounded histories are referenced; owned, bounded data is embedded.

EXERCISE 3.4 — MODEL A PRODUCT CATALOG

Module 3 checkpoint D: three products, one consistent core.

OFFICIAL CHECKPOINT D

20 MIN

labs/day-01/exercises/exercise-3.4-model-a-product-catalog.md

SCENARIO

Common fields: sku, name, description, category, price, active, creation date.

Laptop	processor, memory, storage, screen size
Shoe	size, color, material, gender category
Book	author, ISBN, publisher, language

YOUR TASKS

- 1 List core fields and BSON types
- 2 Decide: top level or attributes
- 3 Write one laptop, shoe and book
- 4 Check names and types match

SOLUTION (AFTER DEBRIEF)

- ✓ sku string · price Decimal128
- ✓ active Boolean · createdAt Date
- ✓ Type-specific data in attributes
- ✓ Queryable by category and attributes.isbn
- ✓ No productPrice beside price

KEY TAKEAWAY A consistent core with nested attributes keeps a varied catalog queryable.

EXERCISE 3.5 — MODEL AN ORDER DOCUMENT

Module 3 checkpoint E: one order aggregate, justified.

OFFICIAL CHECKPOINT E

20 MIN

labs/day-01/exercises/exercise-3.5-model-an-order-document.md

SCENARIO

Find an order by number, show items and purchase-time prices, the shipping address used, payment and fulfillment status, and list a customer's orders.

YOUR TASKS

- 1 Decide items, price, address, customer
- 2 Mark the historical snapshots
- 3 Write the order document
- 4 List the required fields

SOLUTION (AFTER DEBRIEF)

- ✓ Items, prices, address: embedded
- ✓ Customer: customerId reference
- ✓ Money as Decimal128
- ✓ One read by orderNumber
- ✓ Same shape as Lab 2, Step 4

KEY TAKEAWAY An order is a hybrid: reference the customer, embed the purchase facts.

EXERCISE 3.6 — FIND AND CORRECT SCHEMA ANTI-PATTERNS

Module 3 checkpoint F: fix a flawed product document.

OFFICIAL CHECKPOINT F

15 MIN

labs/day-01/exercises/exercise-3.6-correct-schema-anti-patterns.md

SCENARIO

A teammate wrote this product document:

```
{
  productId: "P1001",
  price: "49.99",
  isActive: "yes",
  allReviews: [
    // unlimited growth
  ],
  created: "September 20, 2026"
}
```

YOUR TASKS

- 1 Name every problem you see
- 2 Fix the types
- 3 Move the reviews out
- 4 Choose an _id / sku convention

SOLUTION (AFTER DEBRIEF)

- ✓ String price, text Boolean
- ✓ Unbounded allReviews array
- ✓ Display-text date
- ✓ Decimal128, true, a real Date
- ✓ Reviews in their own collection

KEY TAKEAWAY Most anti-patterns are wrong types or unbounded growth — both are easy to spot once you look.

EXERCISE 3.7 — SELECT A SCHEMA DESIGN PATTERN

Module 3 checkpoint G: match seven requirements to patterns.

OFFICIAL CHECKPOINT G

15 MIN

labs/day-01/exercises/exercise-3.7-select-a-schema-pattern.md

SCENARIO

Choose Attribute, Bucket, Subset, Computed, Extended reference, Outlier or Polymorphic.

- 1 Sensor readings grouped by hour
- 2 Latest three product reviews
- 3 Stored average rating
- 4 Dynamic product specifications
- 5 Customer name copied into order
- 6 Rare products, huge review volume
- 7 Many product types, one collection

YOUR TASKS

- 1 Match requirements 1–4
- 2 Match requirements 5–7
- 3 Give one benefit and one cost
- 4 Explain Bucket and Outlier

SOLUTION (AFTER DEBRIEF)

- ✓ 1 Bucket · 2 Subset
- ✓ 3 Computed · 4 Attribute
- ✓ 5 Extended reference
- ✓ 6 Outlier · 7 Polymorphic

KEY TAKEAWAY Each pattern answers one recurring problem — name the problem first.

EXERCISE 3.8 — DESIGN COLLECTION VALIDATION

Module 3 checkpoint H: a `$jsonSchema` for products.

OFFICIAL CHECKPOINT H

15 MIN

labs/day-01/exercises/exercise-3.8-design-collection-validation.md

SCENARIO

A valid product must contain these fields:

sku	string
name	string
category	string
price	decimal, not negative
active	Boolean
createdAt	date

YOUR TASKS

- 1 Write the required array
- 2 Give each field a `bsonType`
- 3 Add `minimum: 0` to price
- 4 Wrap it in `$jsonSchema`

SOLUTION (AFTER DEBRIEF)

- ✓ Six required field names
- ✓ price: decimal, minimum 0
- ✓ active: bool · createdAt: date
- ✓ bsonType: "object" at the top
- ✓ Extra fields still allowed

KEY TAKEAWAY A validator turns the model's contract into rules the database enforces.

EXERCISE 3.9 — DESIGN A SUPPORT-TICKET DATA MODEL

Module 3 checkpoint I: a complete design challenge.

OFFICIAL CHECKPOINT I

25 MIN

labs/day-01/exercises/exercise-3.9-support-ticket-challenge.md

SCENARIO

Create tickets, assign agents, set status and priority, add comments and tags, show recent comments with the ticket, keep a complete audit history, and search by customer, agent, status and priority.

YOUR TASKS

- 1 List access patterns and collections
- 2 Mark embedded, referenced, unbounded
- 3 Draft one ticket with BSON types
- 4 List validation rules and indexes

SUGGESTED DIRECTION

- ✓ tickets, customers, agents, audit_events
- ✓ Embed: assignee, tags, recentComments
- ✓ Full comments and audit: separate
- ✓ Indexes: customerId, agentId, status, priority

KEY TAKEAWAY The same method works for any application: patterns, relationships, growth, then rules.

LAB 2 — CREATE AND POPULATE TRAINING_STORE

Day 1 hands-on: Steps 1–7 by hand, or the time-boxed load.js path.

OFFICIAL LAB

90 MIN

labs/day-01/lab2/LAB-2-GUIDE.md

SCENARIO

Path A (50 min): the instructor loads the full dataset. Path B (90 min): type the core by hand.

```
# Path A, in PowerShell
mongosh "mongodb://localhost:27017" \
  .\datasets\training_store\load.js
```

YOUR TASKS

- 1 Step 1 Create the four collections
- 2 Steps 2–3 Insert L100, S200, B300, C101
- 3 Step 4 Insert O5001 with references
- 4 Step 5 Query nested fields and arrays
- 5 Step 6 Validate · Step 7 Review

EXPECTED COUNTS

- ✓ Path A: $13 \cdot 6 \cdot 17 \cdot 6$
- ✓ Path B: $3 \cdot 1 \cdot 1 \cdot 0$
- ✓ O5001 customerId = C101's _id
- ✓ Invalid insert rejected
- ✓ XBAD's string price noted

KEY TAKEAWAY By the end of Day 1, training_store exists with a model you can explain field by field.

MODULE 3 SUMMARY AND WHAT'S NEXT

From document structure to a modeled, populated training_store.

YOU CAN NOW

- ✓ Read a document's fields, nesting and _id
- ✓ Choose BSON types that match the meaning
- ✓ Turn requirements into access patterns
- ✓ Embed or reference — and justify it
- ✓ Spot anti-patterns and pick a schema pattern
- ✓ Validate, evolve and query the model

NEXT: MODULE 4 — THE MONGODB QUERY LANGUAGE

- 1 How do I filter, project and sort documents?
- 2 How do I query arrays and embedded fields precisely?
- 3 How do I update and delete documents safely?

DAY 2 Reload training_store with load.js, and bring your Lab 2, Step 7 checklist.

KEY TAKEAWAY Day 1 ends with a designed database; Day 2 starts querying it.